

DATABASE & BACKEND ENGINEERING SUBMISSION

High-Concurrency Movie Booking Backend

Normalized MySQL data model, theatre show discovery, timed seat holds, queue-based burst handling, and idempotent payment webhooks.

Prepared for: BookMyShow ticketing backend scenario

Prepared by: Sameeksha B Hegde

Date: 30 September 2026

Repository: <https://github.com/Sameeksha0509/Bookmyshow-ticketing-backend>

Submission branch: submission/high-concurrency-booking-engine

Verification note: The MySQL schema was created in Workbench, but the seed data and end-to-end API/concurrency test have not been confirmed as successfully run in this environment. Docker Desktop reported virtualization support unavailable. Example result tables below are derived from the supplied seed SQL, not claimed as captured test output.

1. Scope and expected behavior

For a theatre, the application presents seven consecutive dates. Selecting a date lists the scheduled movies, screens, show start/end times, prices, and seat availability. A booking request holds selected seats for a limited period. Concurrent requests for the same seat must not both succeed. Payment events must be safe to retry.

This submission includes the database and backend reference implementation. It does not implement a browser UI matching the reference screenshot.

- Relational entities and keys
- Sample rows and relationships
- Normalization review
- Concurrency, expiry, and payment design
- P1 and P2 SQL
- Test/evidence status

2. Entities, attributes, and table structures

All tables use InnoDB. **PK** = primary key, **FK** = foreign key, **UQ** = unique constraint.

Table / entity	Attributes and types	Keys and purpose
app_users	user_id BIGINT UNSIGNED; email VARCHAR(254); full_name VARCHAR(120); created_at TIMESTAMP(6)	PK user_id; UQ email. Booking customer.
theatres	theatre_id BIGINT UNSIGNED; theatre_name VARCHAR(160); city VARCHAR(80); address VARCHAR(255); created_at TIMESTAMP(6)	PK theatre_id. Theatre location and identity.
screens	screen_id BIGINT UNSIGNED; theatre_id BIGINT UNSIGNED; screen_name VARCHAR(80); capacity SMALLINT UNSIGNED	PK screen_id; FK theatre_id; UQ (theatre_id, screen_name).
seats	seat_id BIGINT UNSIGNED; screen_id BIGINT UNSIGNED; row_label VARCHAR(5); seat_number SMALLINT UNSIGNED; seat_type ENUM(REGULAR, PREMIUM, RECLINER)	PK seat_id; FK screen_id; UQ (screen_id, row_label, seat_number).
movies	movie_id BIGINT UNSIGNED; title VARCHAR(200); duration_minutes SMALLINT UNSIGNED; language_code CHAR(2); certificate VARCHAR(10)	PK movie_id. Movie catalogue attributes.
shows	show_id BIGINT UNSIGNED; screen_id BIGINT UNSIGNED; movie_id BIGINT UNSIGNED; show_date DATE; starts_at DATETIME; ends_at DATETIME; base_price DECIMAL(10,2); status ENUM(SCHEDULED, CANCELLED, COMPLETED)	PK show_id; FKs screen_id/movie_id; UQ (screen_id, starts_at); indexed by screen/date/status/time.
show_seats	show_id BIGINT UNSIGNED; seat_id BIGINT UNSIGNED; price DECIMAL(10,2); status ENUM(AVAILABLE, HELD, BOOKED, BLOCKED); hold_token CHAR(36) NULL; held_until DATETIME(6) NULL; version BIGINT UNSIGNED	Composite PK (show_id, seat_id); FKs to shows and seats. One authoritative inventory row per seat per show.

Table / entity	Attributes and types	Keys and purpose
High-Concurrency Movie Booking Backend — Submission Report bookings	booking_id BIGINT UNSIGNED; booking_reference CHAR(12); user_id BIGINT UNSIGNED; show_id BIGINT UNSIGNED; hold_token CHAR(36); idempotency_key CHAR(36); request_hash CHAR(64); status ENUM(PENDING_PAYMENT, CONFIRMED, EXPIRED, CANCELLED); total_amount DECIMAL(10,2); expires_at DATETIME(6); created_at TIMESTAMP(6)	PK booking_id; FKs user_id/show_id; UQ booking_reference, hold_token, idempotency_key.
booking_seats	booking_id BIGINT UNSIGNED; seat_id BIGINT UNSIGNED; price DECIMAL(10,2)	Composite PK (booking_id, seat_id); FK booking_id. Price snapshot at booking time.
payments	payment_id BIGINT UNSIGNED; booking_id BIGINT UNSIGNED; provider_payment_id VARCHAR(120); amount DECIMAL(10,2); status ENUM(PENDING, SUCCEEDED, FAILED, REFUNDED); provider_created_at DATETIME(6) NULL; updated_at TIMESTAMP(6)	PK payment_id; FK booking_id; UQ booking_id and provider_payment_id.
payment_webhook_events	webhook_event_id BIGINT UNSIGNED; provider_event_id VARCHAR(120); provider_payment_id VARCHAR(120); event_type VARCHAR(80); payload JSON; received_at TIMESTAMP(6); processed_at TIMESTAMP(6) NULL	PK webhook_event_id; UQ provider_event_id; indexed provider_payment_id. Retry/idempotency log.

3. Relationships and sample rows

- One theatre has many screens; each screen has many physical seats and may host many shows.
- Each show is for one movie and one screen. Each show has an inventory row for each physical seat in that screen.
- A user can create many bookings. A booking covers one show and associates one or more seats.
- A booking may have one payment; provider webhook events are recorded independently and deduplicated by provider event id.

Table	Representative sample values from sql/02_seed.sql
app_users	1 aisha@example.com Aisha Rao
theatres	1 PVR Orion Mall Bengaluru 26 Brigade Road
screens	1 theatre 1 Screen 1 capacity 6
seats	1 screen 1 row A seat 1 REGULAR
movies	1 The Last Monsoon 142 min EN U/A
shows	1 screen 1 movie 1 2026-10-01 10:00–12:22 ₹250.00 SCHEDULED
show_seats	show 1 seat 1 ₹250.00 AVAILABLE no hold token/expiry version 0
bookings, booking_seats, payments, payment_webhook_events	Empty in initial seed data; rows are created during booking/payment flows.

4. Normalization

High-Concurrency Movie Booking Backend — Submission Report

- **1NF:** Columns are scalar. Seats are represented as rows rather than repeating arrays or comma-separated values.
- **2NF:** In composite-key relations, attributes depend on the complete key. `show_seats.price` depends on `(show_id, seat_id)`; `booking_seats.price` depends on `(booking_id, seat_id)`.
- **3NF:** Movie, theatre, screen, and physical seat attributes are stored in their own relations rather than copied into show records. Booking-time prices/totals are intentional snapshots.
- **BCNF:** Candidate-key determinants have primary/unique constraints, including email, theatre-screen name, screen-seat position, screen/start time, booking reference/idempotency key, payment/provider payment id, and provider event id. These assertions rely on the business rules documented in `docs/solution.md`.

`booking_seats` does not duplicate `show_id` because it is determined through `booking_id → bookings.show_id`. This avoids a redundant determinant and conflicting show values.

5. High-concurrency and payment handling

Timed seat holds

1. `POST /holds` puts a request on a Redis Stream; consumers process requests asynchronously.
2. A Redis Lua script atomically claims all requested seat keys with a TTL. Redis is a contention-reduction fast path.
3. MySQL remains authoritative: the service opens a transaction, locks `show_seats` rows with `SELECT ... FOR UPDATE` in sorted seat order, validates status and expiry, changes all requested rows to HELD, checks affected-row count, then writes booking and `booking_seats` and commits.
4. If any validation/update fails, SQL rolls back and Redis keys owned by that request token are released. Redis TTL expiry and periodic database cleanup reclaim expired holds.

Pessimistic locking: row locks serialize concurrent writers. **Optimistic metadata:** version increments support clients/caches but are not the final booking guard. The unique inventory key and transaction provide durable correctness.

Payment idempotency

The webhook transaction records a unique provider event id, locks booking/inventory, validates the booking expiry, matching hold ownership and exact amount, writes one payment, confirms booking and seats, then marks the webhook processed. A retry with the same provider event id does not repeat state changes. Optional HMAC-SHA256 signature verification uses `PAYMENT_WEBHOOK_SECRET`.

Indexes and operational design

Indexes cover theatre-to-screen lookup, show date/status/time by screen, movie/date, show-seat status/expiry, bookings by user/date and show/status, and unique payment/webhook identifiers. Redis Streams provide burst buffering; consumer groups spread requests across workers.

6. P2 — shows at a theatre on a given date

High-Concurrency Movie Booking Backend — Submission Report

The following is the executable discovery query included in sql/03_queries.sql. It returns scheduled shows with movie, screen, start/end times, price, and available-seat count.

```
USE booking_engine;
SET @theatre_id = 1;
SET @show_date = '2026-10-01';

SELECT
  t.theatre_id,
  t.theatre_name,
  sh.show_id,
  m.movie_id,
  m.title AS movie_title,
  m.language_code,
  sh.show_date,
  sh.starts_at,
  sh.ends_at,
  sc.screen_name,
  sh.base_price,
  (
    SELECT COUNT(*)
    FROM show_seats ss
    WHERE ss.show_id = sh.show_id
      AND (ss.status = 'AVAILABLE'
        OR (ss.status = 'HELD' AND ss.held_until <= UTC_TIMESTAMP(6)))
  ) AS available_seats
FROM theatres t
JOIN screens sc ON sc.theatre_id = t.theatre_id
JOIN shows sh ON sh.screen_id = sc.screen_id
JOIN movies m ON m.movie_id = sh.movie_id
WHERE t.theatre_id = @theatre_id
  AND sh.show_date = @show_date
  AND sh.status = 'SCHEDULED'
ORDER BY sh.starts_at, sc.screen_name;
```

Expected sample result from the fresh seed file (not a captured execution)

Show	Movie	Screen	Show time	Base price	Available (initial)
1	The Last Monsoon	Screen 1	10:00–12:22, Oct 1	₹250.00	6
4	Paper Boats	Screen 2	11:15–13:00, Oct 1	₹180.00	4
2	Orbit 9	Screen 1	13:30–15:28, Oct 1	₹220.00	6
3	The Last Monsoon	Screen 1	18:30–20:52, Oct 1	₹300.00	6

7. Seven-day theatre date picker

This MySQL 8 recursive CTE returns exactly seven consecutive calendar dates starting today and indicates whether the theatre has scheduled shows for each date.

```
WITH RECURSIVE next_dates (show_date, day_number) AS (
  SELECT CURRENT_DATE, 1
  UNION ALL
  SELECT show_date + INTERVAL 1 DAY, day_number + 1
  FROM next_dates
  WHERE day_number < 7
)
SELECT d.show_date,
  EXISTS (
    SELECT 1
    FROM screens sc
    JOIN shows sh ON sh.screen_id = sc.screen_id
    WHERE sc.theatre_id = @theatre_id
      AND sh.show_date = d.show_date
      AND sh.status = 'SCHEDULED'
  ) AS has_scheduled_shows
```

```
FROM next_dates d  
ORDER BY d.show_date;
```

High-Concurrency Movie Booking Backend — Submission Report

On 2026-09-30, based on the seed rows, October 1 and October 2 have scheduled shows; the other dates in the seven-day window do not. Actual output depends on the day the query is run.

8. P1 SQL and data loading

Run in this order in MySQL Workbench: `sql/01_schema.sql`, then `sql/02_seed.sql`, then the P2/date/seat queries in `sql/03_queries.sql`. Warning: `01_schema.sql` drops and recreates project tables; do not rerun it if data needs to be retained.

The full directly executable schema and sample INSERT statements are included in the repository as `sql/01_schema.sql` and `sql/02_seed.sql`. The following appendix contains the complete schema and seed statements for the P1 deliverable.

Appendix A — P1 schema SQL (part 1)

```

CREATE DATABASE IF NOT EXISTS booking_engine;
USE booking_engine;

SET FOREIGN_KEY_CHECKS = 0;
DROP TABLE IF EXISTS payment_webhook_events;
DROP TABLE IF EXISTS payments;
DROP TABLE IF EXISTS booking_seats;
DROP TABLE IF EXISTS bookings;
DROP TABLE IF EXISTS show_seats;
DROP TABLE IF EXISTS shows;
DROP TABLE IF EXISTS seats;
DROP TABLE IF EXISTS screens;
DROP TABLE IF EXISTS movies;
DROP TABLE IF EXISTS theatres;
DROP TABLE IF EXISTS app_users;
SET FOREIGN_KEY_CHECKS = 1;

CREATE TABLE app_users (
  user_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  email VARCHAR(254) NOT NULL,
  full_name VARCHAR(120) NOT NULL,
  created_at TIMESTAMP(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  PRIMARY KEY (user_id), UNIQUE KEY uq_app_users_email (email)
) ENGINE=InnoDB;

CREATE TABLE theatres (
  theatre_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  theatre_name VARCHAR(160) NOT NULL, city VARCHAR(80) NOT NULL,
  address VARCHAR(255) NOT NULL,
  created_at TIMESTAMP(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  PRIMARY KEY (theatre_id), KEY ix_theatres_city_name (city, theatre_name)
) ENGINE=InnoDB;

CREATE TABLE screens (
  screen_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  theatre_id BIGINT UNSIGNED NOT NULL, screen_name VARCHAR(80) NOT NULL,
  capacity SMALLINT UNSIGNED NOT NULL, PRIMARY KEY (screen_id),
  UNIQUE KEY uq_screens_theatre_name (theatre_id, screen_name),
  CONSTRAINT fk_screens_theatre FOREIGN KEY (theatre_id) REFERENCES theatres(theatre_id),
  CONSTRAINT chk_screens_capacity CHECK (capacity > 0)
) ENGINE=InnoDB;

CREATE TABLE seats (
  seat_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  screen_id BIGINT UNSIGNED NOT NULL, row_label VARCHAR(5) NOT NULL,
  seat_number SMALLINT UNSIGNED NOT NULL,
  seat_type ENUM('REGULAR','PREMIUM','RECLINER') NOT NULL DEFAULT 'REGULAR',
  PRIMARY KEY (seat_id), UNIQUE KEY uq_seats_position (screen_id,row_label,seat_number),
  KEY ix_seats_screen (screen_id),
  CONSTRAINT fk_seats_screen FOREIGN KEY (screen_id) REFERENCES screens(screen_id),
  CONSTRAINT chk_seats_number CHECK (seat_number > 0)
) ENGINE=InnoDB;

CREATE TABLE movies (
  movie_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  title VARCHAR(200) NOT NULL, duration_minutes SMALLINT UNSIGNED NOT NULL,
  language_code CHAR(2) NOT NULL, certificate VARCHAR(10) NOT NULL,
  PRIMARY KEY (movie_id),
  CONSTRAINT chk_movies_duration CHECK (duration_minutes > 0)
) ENGINE=InnoDB;

CREATE TABLE shows (
  show_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  screen_id BIGINT UNSIGNED NOT NULL, movie_id BIGINT UNSIGNED NOT NULL,
  show_date DATE NOT NULL, starts_at DATETIME NOT NULL, ends_at DATETIME NOT NULL,
  base_price DECIMAL(10,2) NOT NULL,
  status ENUM('SCHEDULED','CANCELLED','COMPLETED') NOT NULL DEFAULT 'SCHEDULED',
  PRIMARY KEY (show_id), UNIQUE KEY uqShows_screen_start (screen_id,starts_at),
  KEY ixShows_screen_date_status (screen_id,show_date,status,starts_at),
  KEY ixShows_movie_date (movie_id,show_date),
  CONSTRAINT fkShows_screen FOREIGN KEY (screen_id) REFERENCES screens(screen_id),
  CONSTRAINT fkShows_movie FOREIGN KEY (movie_id) REFERENCES movies(movie_id),
  CONSTRAINT chkShows_time_order CHECK (ends_at > starts_at),
  CONSTRAINT chkShows_price CHECK (base_price >= 0)
) ENGINE=InnoDB;

CREATE TABLE show_seats (
  show_id BIGINT UNSIGNED NOT NULL, seat_id BIGINT UNSIGNED NOT NULL,
  price DECIMAL(10,2) NOT NULL,
  status ENUM('AVAILABLE','HELD','BOOKED','BLOCKED') NOT NULL DEFAULT 'AVAILABLE',
  hold_token CHAR(36) NULL, held_until DATETIME(6) NULL,
  version BIGINT UNSIGNED NOT NULL DEFAULT 0,

```

PRIMARY KEY (show_id,seat_id), KEY ix_show_seats_available (show_id,status,held_until),
KEY ix_show_seats_hold_token (hold_token),
CONSTRAINT fk_show_seats_show FOREIGN KEY (show_id) REFERENCES shows(show_id),
CONSTRAINT fk_show_seats_seat FOREIGN KEY (seat_id) REFERENCES seats(seat_id),
CONSTRAINT chk_show_seats_price CHECK (price >= 0),
CONSTRAINT chk_show_seats_hold_data CHECK (
 (status='HELD' AND hold_token IS NOT NULL AND held_until IS NOT NULL) OR status<>'HELD'
)
) ENGINE=InnoDB;

Appendix A — P1 schema SQL (part 2)

```

CREATE TABLE bookings (
  booking_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  booking_reference CHAR(12) NOT NULL, user_id BIGINT UNSIGNED NOT NULL,
  show_id BIGINT UNSIGNED NOT NULL, hold_token CHAR(36) NOT NULL,
  idempotency_key CHAR(36) NOT NULL, request_hash CHAR(64) NOT NULL,
  status ENUM('PENDING_PAYMENT','CONFIRMED','EXPIRED','CANCELLED') NOT NULL DEFAULT 'PENDING_PAYMENT',
  total_amount DECIMAL(10,2) NOT NULL, expires_at DATETIME(6) NOT NULL,
  created_at TIMESTAMP(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  PRIMARY KEY (booking_id), UNIQUE KEY uq_bookings_reference (booking_reference),
  UNIQUE KEY uq_bookings_hold_token (hold_token), UNIQUE KEY uq_bookings_idempotency (idempotency_key),
  KEY ix_bookings_user_created (user_id,created_at), KEY ix_bookings_show_status (show_id,status),
  CONSTRAINT fk_bookings_user FOREIGN KEY (user_id) REFERENCES app_users(user_id),
  CONSTRAINT fk_bookings_show FOREIGN KEY (show_id) REFERENCES shows(show_id),
  CONSTRAINT chk_bookings_total CHECK (total_amount >= 0)
) ENGINE=InnoDB;

CREATE TABLE booking_seats (
  booking_id BIGINT UNSIGNED NOT NULL, seat_id BIGINT UNSIGNED NOT NULL,
  price DECIMAL(10,2) NOT NULL, PRIMARY KEY (booking_id,seat_id),
  CONSTRAINT fk_booking_seats_booking FOREIGN KEY (booking_id) REFERENCES bookings(booking_id),
  CONSTRAINT chk_booking_seats_price CHECK (price >= 0)
) ENGINE=InnoDB;

CREATE TABLE payments (
  payment_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  booking_id BIGINT UNSIGNED NOT NULL, provider_payment_id VARCHAR(120) NOT NULL,
  amount DECIMAL(10,2) NOT NULL,
  status ENUM('PENDING','SUCCEEDED','FAILED','REFUNDED') NOT NULL DEFAULT 'PENDING',
  provider_created_at DATETIME(6) NULL,
  updated_at TIMESTAMP(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6) ON UPDATE CURRENT_TIMESTAMP(6),
  PRIMARY KEY (payment_id), UNIQUE KEY uq_payments_booking (booking_id),
  UNIQUE KEY uq_payments_provider_id (provider_payment_id),
  CONSTRAINT fk_payments_booking FOREIGN KEY (booking_id) REFERENCES bookings(booking_id),
  CONSTRAINT chk_payments_amount CHECK (amount >= 0)
) ENGINE=InnoDB;

CREATE TABLE payment_webhook_events (
  webhook_event_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  provider_event_id VARCHAR(120) NOT NULL, provider_payment_id VARCHAR(120) NOT NULL,
  event_type VARCHAR(80) NOT NULL, payload JSON NOT NULL,
  received_at TIMESTAMP(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6), processed_at TIMESTAMP(6) NULL,
  PRIMARY KEY (webhook_event_id), UNIQUE KEY uq_webhook_provider_event (provider_event_id),
  KEY ix_webhook_payment (provider_payment_id)
) ENGINE=InnoDB;

DROP PROCEDURE IF EXISTS release_expired_holds;
DELIMITER $$
CREATE PROCEDURE release_expired_holds()
BEGIN
  UPDATE show_seats SET status='AVAILABLE',hold_token=NULL,held_until=NULL,version=version+1
  WHERE status='HELD' AND held_until <= UTC_TIMESTAMP(6);
  UPDATE bookings SET status='EXPIRED'
  WHERE status='PENDING_PAYMENT' AND expires_at <= UTC_TIMESTAMP(6)
  AND NOT EXISTS (SELECT 1 FROM payments p
    WHERE p.booking_id=bookings.booking_id AND p.status='SUCCEEDED');
END$$
DELIMITER ;

```

Appendix B — P1 sample data SQL

```
USE booking_engine;

INSERT INTO app_users (email, full_name) VALUES
('aisha@example.com', 'Aisha Rao'),
('ben@example.com', 'Ben Carter');

INSERT INTO theatres (theatre_name, city, address) VALUES
('PVR Orion Mall', 'Bengaluru', '26 Brigade Road'),
('Cinepolis Seasons', 'Pune', 'Magarpatta Road');

INSERT INTO screens (theatre_id, screen_name, capacity) VALUES
(1, 'Screen 1', 6), (1, 'Screen 2', 4), (2, 'Screen 1', 4);

INSERT INTO seats (screen_id, row_label, seat_number, seat_type) VALUES
(1, 'A', 1, 'REGULAR'), (1, 'A', 2, 'REGULAR'), (1, 'A', 3, 'REGULAR'),
(1, 'B', 1, 'PREMIUM'), (1, 'B', 2, 'PREMIUM'), (1, 'B', 3, 'PREMIUM'),
(2, 'A', 1, 'REGULAR'), (2, 'A', 2, 'REGULAR'), (2, 'B', 1, 'PREMIUM'), (2, 'B', 2, 'PREMIUM'),
(3, 'A', 1, 'REGULAR'), (3, 'A', 2, 'REGULAR'), (3, 'B', 1, 'PREMIUM'), (3, 'B', 2, 'PREMIUM');

INSERT INTO movies (title, duration_minutes, language_code, certificate) VALUES
('The Last Monsoon', 142, 'EN', 'U/A'), ('Orbit 9', 118, 'HI', 'U/A'), ('Paper Boats', 105, 'EN', 'U');

INSERT INTO shows (screen_id, movie_id, show_date, starts_at, ends_at, base_price) VALUES
(1, 1, '2026-10-01', '2026-10-01 10:00:00', '2026-10-01 12:22:00', 250.00),
(1, 2, '2026-10-01', '2026-10-01 13:30:00', '2026-10-01 15:28:00', 220.00),
(1, 1, '2026-10-01', '2026-10-01 18:30:00', '2026-10-01 20:52:00', 300.00),
(2, 3, '2026-10-01', '2026-10-01 11:15:00', '2026-10-01 13:00:00', 180.00),
(2, 2, '2026-10-02', '2026-10-02 19:00:00', '2026-10-02 20:58:00', 240.00),
(3, 1, '2026-10-01', '2026-10-01 20:00:00', '2026-10-01 22:22:00', 200.00);

INSERT INTO show_seats (show_id, seat_id, price)
SELECT sh.show_id, se.seat_id,
       sh.base_price + CASE se.seat_type
                        WHEN 'PREMIUM' THEN 50.00 WHEN 'RECLINER' THEN 120.00 ELSE 0.00 END
FROM shows sh JOIN seats se ON se.screen_id=sh.screen_id;
```

This appendix is compactly formatted for the report. The authoritative executable SQL is the matching repository files `sql/01_schema.sql` and `sql/02_seed.sql`.

Appendix C — Verification plan and evidence

Case	Expected outcome	Status in this submission
32 concurrent requests for one seat	Exactly one hold succeeds; others conflict	Test implemented; execution not verified (Docker engine unavailable).
Same idempotency key, same payload	Existing booking returned	Test implemented; execution not verified.
Same idempotency key, changed payload	Conflict/rejection	Test implemented; execution not verified.
Payment amount mismatch	Reject event, do not confirm booking	Test implemented; execution not verified.
Duplicate provider event	Accept retry without duplicate state changes	Test implemented; execution not verified.

Screenshots to add after running locally

The following Workbench evidence images were supplied in the repository README and are reproduced here. They illustrate the show list, date picker, seat selection, and table list.

Evidence 1: Scheduled shows

SQL File 14*SQL File 15*02_seedSQL File 17*03_queriesSQLAdditions

Limit to 1000 rowsJump to

```
1 • USE booking_engine;
2
3 -- Set the theatre and local calendar date requested by the caller
4 • SET @theatre_id = 1;
5 • SET @show_date = '2026-10-01';
6
7 -- P2: every scheduled show and its time at this theatre on this date
8 • SELECT
9     t.theatre_id,
10    t.theatre_name,
11    sh.show_id,
12    m.movie_id,
13    m.title AS movie_title,
14    m.language_code,
```

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Context HelpSnippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 51	20:45:26	INSERT INTO shows (screen_id, movie_id, show_date, starts_at, ends_at, base_p...	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0	0.000 sec
✓ 52	20:45:26	INSERT INTO show_seats (show_id, seat_id, price) SELECT sh.show_id, se.seat_id, ...	30 row(s) affected Records: 30 Duplicates: 0 Warnings: 0	0.016 sec
✓ 53	20:47:01	USE booking_engine	0 row(s) affected	0.000 sec
✓ 54	20:47:01	SHOW TABLES	11 row(s) returned	0.015 sec / 0.000 sec
✓ 55	20:48:11	SET @show_date = '2026-10-01'	0 row(s) affected	0.000 sec

Evidence 2 - Seven-day date picker

SQL File 14* SQL File 15* 02_seed SQL File 17* 03_queries SQLAdditions

Limit to 1000 rows

33 AND sh.status = 'SCHEDULED'

34 ORDER BY sh.starts_at, sc.screen_name;

35

36 -- Date picker: seven consecutive dates starting today, including

37 WITH RECURSIVE next_dates (show_date, day_number) AS (

38 SELECT CURRENT DATE, 1

Result Grid Filter Rows: Export: Wrap Cell Content: Result Grid Form Editor

show_date	has_scheduled_shows
2026-09-30	0
2026-10-01	0
2026-10-02	0
2026-10-03	0
2026-10-04	0
2026-10-05	0

Result 1 x Read Only Context Help Snippets

Output Action Output

#	Time	Action	Message	Duration / Fetch
52	20:45:26	INSERT INTO show_seats (show_id, seat_id, price) SELECT sh.show_id, se.seat_id, ...	30 row(s) affected Records: 30 Duplicates: 0 Warnings: 0	0.016 sec
53	20:47:01	USE booking_engine	0 row(s) affected	0.000 sec
54	20:47:01	SHOW TABLES	11 row(s) returned	0.015 sec / 0.000 sec
55	20:48:11	SET @show_date = '2026-10-01'	0 row(s) affected	0.000 sec
56	20:51:08	WITH RECURSIVE next_dates (show_date, day_number) AS (SELECT CURRE...	7 row(s) returned	0.015 sec / 0.000 sec

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Evidence 3 - Seat selection Submission Report

SQL File 13* SQL File 14* SQL File 15* 02_seed SQL File 17* 03_queries SQLAdditions

Limit to 1000 rows

70 FROM show_seats ss
71 JOIN seats se ON se.seat_id = ss.seat_id
72 WHERE ss.show_id = @show_id
73 ORDER BY se.row_label, se.seat_number;
74

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Result Grid

show_id	seat_id	row_label	seat_number	seat_type	price	effective_sta
1	1	A	1	REGULAR	250.00	AVAILABLE
1	2	A	2	REGULAR	250.00	AVAILABLE
1	3	A	3	REGULAR	250.00	AVAILABLE
1	4	B	1	PREMIUM	300.00	AVAILABLE
1	5	B	2	PREMIUM	300.00	AVAILABLE
1	6	B	3	PREMIUM	300.00	AVAILABLE

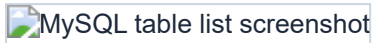
Result 4 x Read Only Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
56	20:51:08	WITH RECURSIVE next_dates (show_date, day_number) AS (SELECT CURRE...	7 row(s) returned	0.015 sec / 0.000 sec
57	20:52:39	WITH RECURSIVE next_dates (show_date, day_number) AS (SELECT CURRE...	7 row(s) returned	0.000 sec / 0.000 sec
58	20:53:16	SET @show_id = 1	0 row(s) affected	0.000 sec
59	20:53:28	WITH RECURSIVE next_dates (show_date, day_number) AS (SELECT CURRE...	7 row(s) returned	0.016 sec / 0.000 sec
60	20:56:02	SELECT ss.show_id, ss.seat_id, se.row_label, se.seat_number, se.sea...	6 row(s) returned	0.000 sec / 0.000 sec

Evidence 4 — Database tables



The screenshots above are repository-supplied Workbench images. They are not evidence that the concurrency/webhook integration test passed. Docker Desktop could not start because virtualization support was unavailable, so that test remains unverified and must be run before claiming it passed.

Repository PR: <https://github.com/Sameeksha0509/Bookmyshow-ticketing-backend/compare/main...submission/high-concurrency-booking-engine?expand=1>

Local run: Start Docker Desktop with virtualization enabled; then run `docker compose up --build -d`, install `tests/requirements.txt`, and run `python tests/concurrency_test.py`.